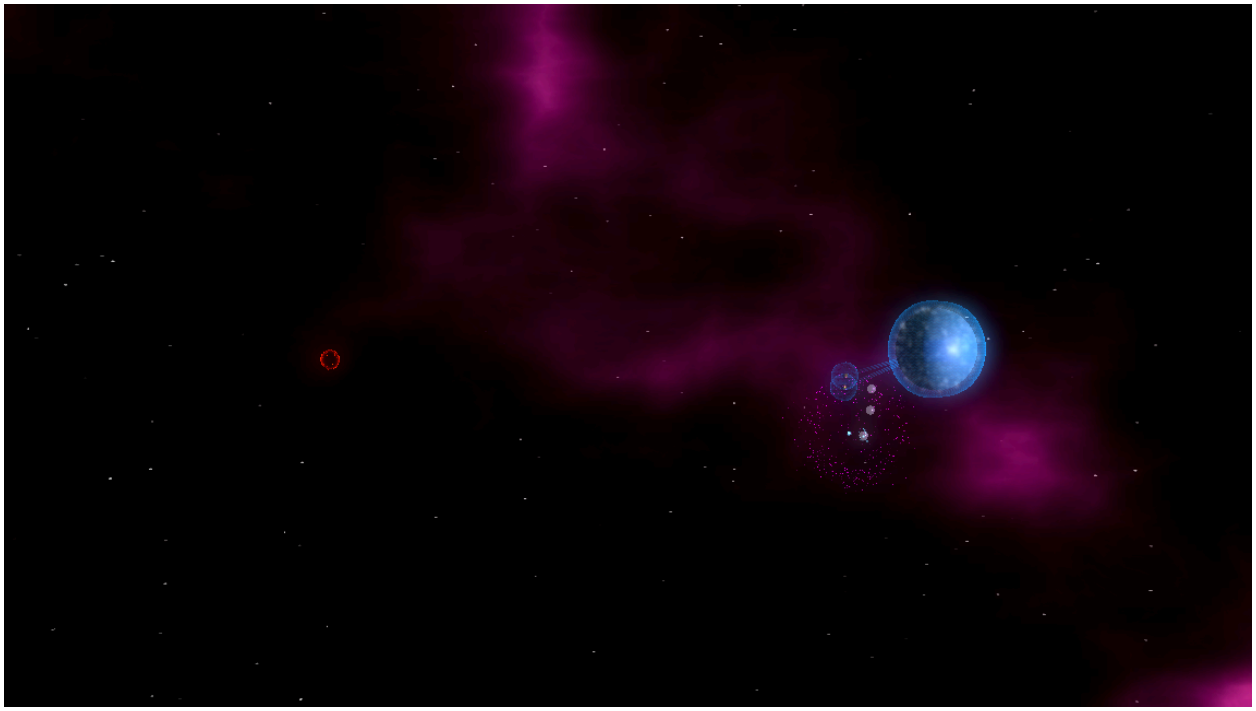
Murphy Potts

Technical Portfolio

Rogue-lite Game - Godot3D

Gameplay: https://youtu.be/kisZar7x1E4

Play as a singularity in the universe. Absorb planets, stars, and many other entities as you grow. Face off against planetary defenses like shields and stabilizing bombs.



Player singularity orbiting a shielded planet

Technical Challenge:

Containing the entire game within a single “treadmill” root scene to preserve immersion (no loading screens), while also maintaining great performance for gameplay.

Engineering solution:

Utilizing Godot engines “load_threaded_request” and “load_threaded_get” with a singleton script to keep memory usage to a minimum. This creates the “treadmill” architecture, dynamic asset lifecycle management without loading screens.

Result:

Achieved seamless world-streaming with zero frame-stutter during scene transitions, maintaining a consistent 60 FPS and optimized memory footprint.

Metroidvania Game - Godot3D

Gameplay: https://youtu.be/kAJJNN7dHA4

Survive a lovecraftian nightmare by shooting your way out. Creatures continue to evolve as you defeat them, creating challenge and unpredictability for the player.

“Eyeball cluster” enemies evolve into flying eyes when clusters are shot

Technical Challenge:

When shooting enemy creatures, state-driven entity transformation evolves creatures.

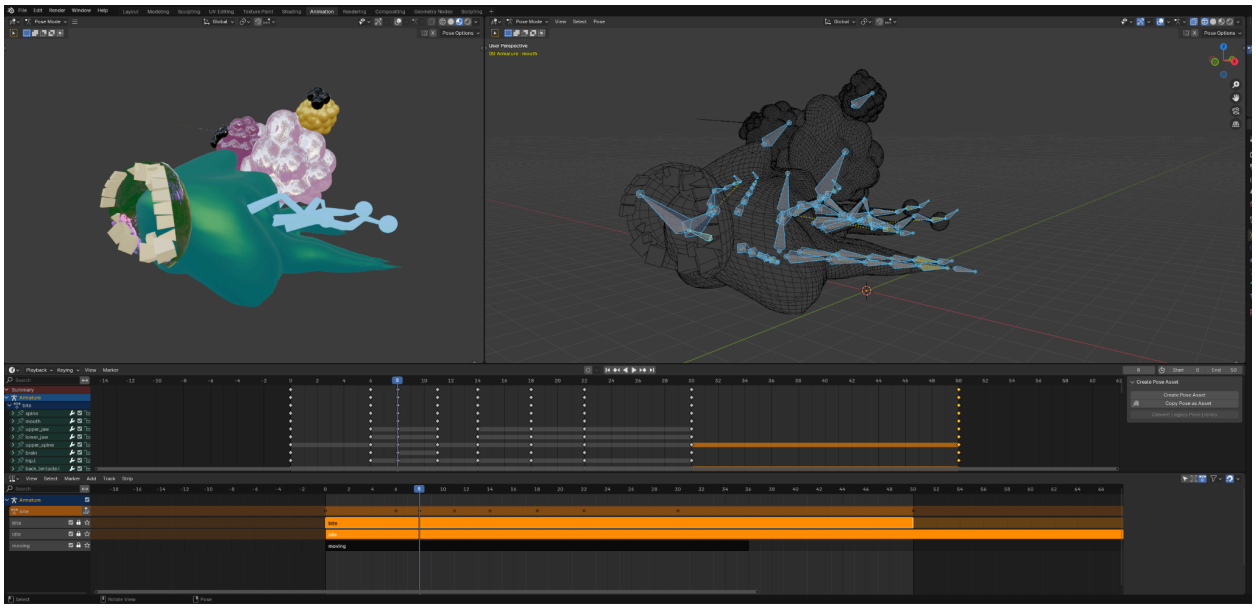
Engineering solution:

Use the architecture pattern of Object Pooling. Create a reusable enemy scene that accepts a dynamic number of scenes to instantiate when triggered by gameplay, enacting state-driven entity transformation.

Result:

Implemented an Object Pooling pattern to manage entity lifecycles, enabling dynamic enemy transformations without the performance overhead of repeated instantiation and garbage collection.

Technical Asset Pipeline - Blender to Godot3D



Saving animation data to NLA tracks for Godot

Process:

Optimized high-poly sculpts into game-ready geometry via manual retopology and UV unwrapping, ensuring efficient texture memory usage and vertex counts.

Animation System:

Developed a modular rigging system with custom constraints, utilizing NLA (Non-Linear Animation) tracks to export clean, reusable animation data for the Godot state machine.

Result:

Created a seamless "Art-to-Engine" workflow that preserves animation fidelity while minimizing the performance overhead of complex skeletal transforms.